Analiza Algoritmilor

SERIA CA

Temă de casă 2014-2015

-partea a-II-a-

Problema 1 – 10p

Se considera algoritmul:

Algoritm

Input: a, b ∈ ℕ
Output: c ∈ ℕ

```
1: c, d ← 0
2: e ← 1
3: while (a > 0) ∨ (b > 0) ∨ (d > 0) then
4:     f ← a % 2
5:     g ← b % 2
6:     if f ⊕ g ⊕ d then // ⊕ reprezinta operatorul logic XOR
7:         c ← c + e
8:     end if
9:     d1 ← (f ∧ g)  // ∧ reprezinta operatorul logic SI
10:    d2 ← (g ∧ d)
11:    d3 ← (f ∧ d)
12:    d ← d1 ∨ d2 ∨ d3 // ∨ reprezinta operatorul logic SAU
13:    e ← 2*e
14:    a ← a / 2
15:    b ← b / 2
16: end while
17: return c
```

Demonstrati corectitudinea pentru urmatorul invariant care explica functionarea algoritmului de mai sus:

$$(a_i + b_i + d_i)*e_i + c_i = a_0 + b_0, \forall i \geq 0$$

unde a_0, b_0 sunt valorile inainte de executia *while,* iar a_i, b_i, d_i, e_i, c_i sunt valorile la sfarsitul iteratiei *i* a buclei.

Problema 2.1 – 5p

Pentru tipul ***T LIST*** definit prin constructorii de baza:
[] : -> T LIST
[a] : T -> T LIST
a::l (sau cons(a, x)) : T x T LIST -> T LIST (adica *a* adaugat la inceputul listei *l*)

Definiti functiile:
size(l) : T LIST -> N – intoarce numarul de elemente din lista *l*
reverse(l) – inverseaza elementele din lista *l*
unique(l) : T LIST -> T LIST – intoarce o lista formata din elementele cu aparitii unice in lista *l*
repeated(l) : T LIST -> T LIST – intoarce o lista formata din elementele care se repeta in lista *l*

max?(e, l) : T x T LIST -> Bool – intoarce *true* daca elementul *e* este elementul maxim al listei *l* si intoarce *false* altfel

Problema 2.2 – 10p

Folosind inductia structurala, demonstrati urmatoarea proprietate pentru oricare lista *l* de tip T LIST:

size(unique(l)) == size(l) -> unique(l) == l

Problema 2.3 – 13p

Folosind inductia structurala, demonstrati urmatoarea proprietate pentru oricare lista *l* de tip T LIST:

max?(e, l) -> max?(e, unique(reverse(l))) || max?(e, repeated(reverse(l)))

Problema 3.1 – 8+2p

Scrieţi un algoritm nedeterminist pentru urmatoarea problema: *Se da un graf neorientat G = (V, E) si un numar intreg pozitiv k. Exista o partitie (S, V \ S) a multimii V astfel incat multimea E' = {(u,v) ∈ E | u ∈ S, v ∈ V \ S} are card(E') ≥ k?*

Problema 3.2 – 8+2p

Scrieţi un algoritm nedeterminist pentru urmatoarea problema: *Se dau doua grafuri neorientate G1 = (V1, E1), G2 = (V2, E2) si un numar natural k. Exista un graf G3 = (V3, E3), card(V3) = k astfel incat G3 este subgraf atat pentru G1 cat si pentru G2?*

Problema 3.3 – 8 +2p

Scrieţi un algoritm nedeterminist pentru urmatoarea problema: *Se da o multime de n perechi de numere intregi pozitive S = {(v_i , w_i) | i = 1...n} si doua numere intregi pozitive V si W. Exista un subset C ⊆ S astfel incat* $\sum\{v_i \mid (v_i, w_i) \in C\} \geq V$ *si* $\sum\{w_i \mid (v_i, w_i) \in C\} \leq W$*?*

Determinati si justificati complexitatea temporala si spatiala pentru algoritmii nedeterministi construiti.

Problema 4 – 10p

Demonstrati ca problema *3SAT* se reduce polinomial la problema *3-Colorare*.

Problema *3SAT: Fiind data o formula F in forma normala conjunctiva, cu proprietatea ca fiecare disjunctie contine maxim 3 literali, este F satisfiabila?*

Problema *3-Colorare: Se dau trei culori si un graf neorientat G=(V, E). Este posibil sa coloram nodurile din graf folosind cele trei culori de intrare astfel incat nicio muchie din graf nu are nodurile colorate in aceeasi culoare?*

Problema 5 – 12p

Demonstrati ca problema *Independent Set* se reduce polinomial la problema *k-Clica*.

Problema *Independent Set: Se da un graf neorientat G=(V, E) si un numar natural k. Exista un subset V' de noduri astfel incat card(V')=k, iar fiecare muchie are cel mult un capat in V'?*

Problema *k-Clica: Se da un graf neorientat G1=(V1, E1) si un numar natural k. Exista o k-clica in graful G1? O k-clica intr-un graf G1=(V1, E1) este definita ca un subgraf G2=(V2, E2) complet al lui G1, astfel incat card(V2)=k.*